Lecture One — Construction of Correct Programs

Study handout — text extracted from the original slides (wording preserved). Logic symbols fixed (∧ ∨ ¬ ⇒ ⇐ ≡ ≠ ⊥). About half the paper or less. 76 slides. Black text on white.

How to use: skim slide titles, study bullets in order, read formulas as written.

Slide 1 — FGSSR Fall 2025

Lecture One
CONSTRUCTION OF
CORRECT PROGRAM

Slide 2 — WHY CORRECT PROGRAM?

• Disaster is an understatement for any organization that has incurred losses due to a catastrophic software glitch.
• While technology and innovative applications have empowered brands, enterprises have recorded numerous disabling instances.
• Software disasters have the potential to cause widespread disruptions and highlight the importance of rigorous verification, validation and quality control measures in preventing catastrophic software failures.
• Any application's unexpected crashes and data inconsistencies were a direct result of the Failure In Designing Correct Software, revealing significant gaps in the quality assurance process.
• https://raygun.com/blog/costly-software-errors-history/

Slide 3 — STATISTICS FROM 2025 AND LATE 2024:

• Over half (52.7%) of software projects exceed their budgets by at least 89%.
• Only about 29% of software development projects achieve their goals by delivering on time and within budget.
• Only 52% of software projects pass quality tests after release
• Over 80% of project managers admit to knowingly shipping flawed software, according to data from early 2025.
• Asignificant portion of software projects face challenges, with one report stating that 70% of project failures occur during digital transformations.

Slide 4

• The entirety of 2025 are not available, several reports highlight the prevalence and impact of software failures.
• Almost half (47%) of unsuccessful software projects are attributed to poor project management.
• Over half (52.7%) of software projects exceed their initial budgets by at least 89%.
• Only 52% of software projects pass quality tests after release.

Slide 5

• Between 40% and 50% of projects are delivered late.
• 78% of projects experience scope creep, often leading to failure.
• Arecent study found that 67% of software teams cannot build and test their development environment within the scheduled period.
• 64% of infrastructure code deployments still rely on manual steps, and 50% of application deployments still rely on manual processes.

Slide 6 — MAJOR SOFTWARE BUGS IN 2025

• Major software bugs in 2025 have not been tied to a single company
• Largely categorized as security vulnerabilities that have impacted many organizations.
• As of September 2025, cybersecurity reports indicate a general increase in attack sophistication and volume.
• Several large-scale data breaches often tracing back to common weaknesses in third-party software.

Slide 7 — MAJOR INCIDENTS IN 2025

• Several major incidents in 2025 involved exploiting software vulnerabilities, impacting multiple companies and millions of individuals.

Incidents tied to vulnerabilities
• Microsoft SharePoint vulnerabilities: Chinese-linked threat actors exploited multiple zero-day vulnerabilities in on-premises Microsoft SharePoint servers.
• Attackers used these bugs to compromise systems in hundreds of organizations, including US government agencies.
• SonicWall SMA attacks: Threat actors exploited a zero-day RCE vulnerability (Remote Code Execution is a type of security vulnerability that allows attackers to run arbitrary code on a remote machine)
in SonicWall's Secure Mobile Access (SMA) products, which was confirmed by US Cybersecurity and Infrastructure Security Agency (CISA).

Slide 8

• VMware vulnerabilities (allows local attackers with administrative privileges on a virtual machine to achieve code execution on the host system to leak information): In March, Broadcom released security updates for actively exploited flaws in VMware that affected ESXi, (ESX and ESXI are thin lightweight Hyper Visor OS) Workstation.
• These vulnerabilities, including a race condition bug, were exploited in the wild before public disclosure.

Slide 9

• PHP vulnerability: A critical RCE vulnerability affecting Windows -based PHP installations was widely exploited in March 2025, impacting many organizations in the US.
• Magecart attacks: E -skimming attacks on e-commerce websites that use malicious JavaScript to steal customer Payment information. The number of affected e-commerce domains remained high in early 2025.

Slide 10 — 2024 SHARE OF SOFTWARE FAILURES

• From big tech companies to social media platforms and stock exchanges, here are seven major tech glitches from 2024 that remind us how much we rely on technology.

Slide 11 — META SERVICES

• On March 5, 2024, Meta faced a major outage that disrupted access to its popular platforms, including Facebook and Instagram, for several hours.
• Meta acknowledged the outage and apologized for the inconvenience
• While some reports suggested widespread network issues, further analysis indicated that the problem was likely related to a backend issue within Meta’s infrastructure, possibly involving authentication or core system functions.

Slide 12 — NYSE:

THE NEW YORK STOCK EXCHANGE
• On June 3, 2024, a technical issue with industry-wide price bands from the Consolidated Tape Association’s Securities Information Processor (CTA SIP) caused trading halts for several stocks on NYSE Group exchanges.
• The error led trading platforms to mistakenly show a nearly 100%
drop in Berkshire Hathaway’s share price, making it appear as though Warren Buffett’s (CEO)company had lost almost all its value.
• Trading in Berkshire Hathaway stock was paused for about two hours before resuming.

Slide 13 — MICROSOFT AND CROWDSTRIKE

• One of the biggest tech failure was a glitch with Microsoft and CrowdStrike that affected 8.5 million Windows devices, causing systems to crash and disrupting businesses, governments, and people worldwide.

Slide 14

• On July 19, 2024, Microsoft users experienced an outage due to a mistake made by CrowdStrike, a cybersecurity company. They released a faulty software update that caused Windows systems using CrowdStrike's Falcon sensor (endpoint detection and response" (EDR) software) to crash, leading to widespread disruptions.
• As a result, the 911 emergency lines were down in several U.S. states, the Federal Aviation Administration reported that some International airlines reported disruptions.
• the outage affected many important services worldwide, including air travel, healthcare, financial markets, and media. This event highlights how connected and vulnerable our systems are when something goes wrong.
• On the day of the outage, Microsoft's stock suffered 3.6% decline, CrowdStrike's stock also fell by 17.9%. Besides financial losses, these tech companies also faced public embarrassment due to the glitch.

Slide 15 — SIX SWISS EXCHANGE

• On July 31, 2024, trading on the SIX Swiss Exchange was halted for several hours due to technical problems.
• As a result, the SMI, (Six main index), dropped by 3.6%
• the disruptions in essential services and halted financial transactions show how a failure in tech infrastructure can cause major problems in everyday life and business.
• which emphasized the importance of having strong backup plans and diverse technology solutions to reduce risks.

Slide 16 — ELON MUSK INTERVIEW

• On August 12, 2024, a technical glitch on X caused a 40 -minute delay in the much-anticipated interview between former US President Donald Trump and X owner Elon Musk. Musk attributed the issue to a cyberattack.
• The glitch prevented many users from accessing the X Spaces audio stream through the shared link, leaving 765,000 listeners with only music until the interview finally started.

Slide 17 — NETHERLANDS 'S MINISTRY OF DEFENSE

• On August 28, 2024, air traffic at Eindhoven Airport, the second-largest airport in the Holand- was halted for several hours due to IT network issues at the Dutch Ministry of Defense.
• The ministry clarified that there was no evidence of malicious intent, stating that the disruptions were caused by a software bug.
• This issue led to login problems for Ministry of Defense employees and affected other government services as well.

Slide 18 — HGC 'S INTERNET (Uncontrolled Connection…

• In February 2024, HGC Global Communications (internet service provider of Hong Kong) experienced major internet service disruptions after four of its underwater cables in the Red Sea were damaged.
• This severely affected communication networks across the Middle East, impacting 25% of their internet traffic.
• The incident highlighted the fragility of global internet infrastructure and the widespread effects on businesses and individuals who rely on consistent online connectivity.

Slide 19 — TO MAKE LONG STORY SHORT

• Between 2023 and 2025, technology failures caused by vendor updates, infrastructure breakdowns, and deliberate attacks resulted in operational, reputational, and financial damage.
• In 2025, UK manufacturers face increased risks of expensive software failures, with six in ten expecting major outages. This is linked to the pressure to accelerate digital delivery, leaving quality assurance behind.
• Poor software quality is estimated to cost UK manufacturers between £390,000 and £773,000 annually

Slide 20 — CAUSES OF SOFTWARE FAILURES AND DEFECTS

• Lack of collaboration and code/test coverage
• Poor testing frameworks, inadequate defect management, and not testing under real user conditions.
• Unclear requirements, lack of communication, poor project management, unrealistic expectations, and insufficient user involvement.
• The shift to cloud-native architectures like micro -services and Kubernetes introduces operational complexities.
• Developer burnout and high developer turnover are also concerns, with increasing complexity and stress levels identified as contributing factors.
• Cloud misconfigurations: Human errors, such as leaving cloud storage buckets publicly accessible, that can lead to massive data leaks.

Slide 21 — SOFTWARE FAILURE SHOULD NOT BE

TOLERATED
• Tech outages can disrupt business operations, financial markets, and daily life, exposing the economic risks tied to our dependence on digital infrastructure.
• Verification and validation are crucial for improving system reliability, efficiency, and resilience, making them key to preventing and managing such tech glitches (Fail Safe).

Slide 22 — COURSE OBJECTIVE

• The course tackle the verification problem by constructing a correct program step at a tim e, applying theory of Program m ing
• We are concerned in:
1. Proving each step as we develop it.
2. Modification of programs (with proof that the modification is correct.)

Slide 23 — APPLICATIONS: WHICH?

• Communication Protocols
• Processors (CPUs)
• Kernel of secured distributed O/S
• Compilers
• Safety critical: medical systems, nuclear control
• Railway, aerospace controls
• Telephone and internet switching systems
• Any critical systems (that cause loses in human lives, properties, Money, …

Slide 24 — Who Wants Th eory Of Program m in g,… Why?

• One answer is that a mathematical theory gives a much greater degree of precision by providing a method of calculation.
The software industry has an overwhelming experience of buggy programs to support that statement.
• Another answer is that a theory provides a kind of understanding.
Programming changes from an art to a science when we learn to understand programs in the same way we understand mathematical theorems.

Slide 25

• With a scientific outlook, we change our view of the world.
• It is a valuable part of education for anyone.
• Professional Software engineers, to be worthy of the name, must know and apply a theory of programming.
• The subject of this course sometimes goes by the names
“Programming Methodology ”, “Science Of Programming ”, “Logic of Programming ”, “Theory of Programming ”, “Formal Methods of Program Development ”, “Programming From Specifications”, or

“VERIFICATION ”.

Slide 26

• Theory Of Program m ing concerns those aspects of programming that are amenable to mathematical proof.
• Agood theory helps us to write precise specifications, and to design programs whose executions provably satisfy the specifications.

Slide 27 — Our Theory In this theory

• Aspecification is just a binary expression.
• Refinement is just implication.
• it is, also, more comprehensive, because it is applicable to:
• both terminating and nonterminating computation,
• both sequential and concurrent computation,
• both stand-alone and interactive computation.

Slide 28 — The Relevant Mathematics

• Formal Methods of Software Engineering means the use of mathematics as an aid to writing programs.
• Let us first introduce the relevant mathematics (only what is needed).
• Most of you may have already learned most of this math in other courses.
• Just to be sure we're all using the same notations.
• If you look at the handout, you'll see all the notations used in the formal verification methodology.

Slide 29

Required Math Binary Number theory Character theory theory Domain Functions effect over the domain of local variables

Slide 30 — Course Contents

(No body text on this slide.)

Slide 31

• BASIC THEORIES: Binary Theory, Number Theory, Character Theory.
• BASIC DATA STRUCTURES: Bunch Theory, Set Theory, String Theory, and List Theory.
• FUNCTION THEORY: Functions, Function Notations, Scope, Substitution, and Quantifiers.
• PROGRAM THEORY: Specifications, Specification Notations, Specification Laws, Programs, Program Development, and Refinement Laws.
• TIME: Real Time, Recursive Time, Termination, Soundness and Completeness.

Slide 32 — Binary Theory

• Binary Theory, is also called Boolean algebra, basic logic, or propositional calculus.
• Some binary expressions are called theorems, and others are antitheorems.
• Binary expressions represent anything that comes in two kinds, theorems represent one kind, and antitheorems represent the other.
• One may think that statements about the world come in two kinds, (true statements and false statements)

Slide 33 — Binary Theorems

• Here are the two simplest binary expressions:
• T, ⊥ (True/False, power/ground, innocent/guilty, top/bottom)

Slide 34 — Logical Notations Might have one, two or…

(No body text on this slide.)

Slide 35 — Logical Notations: ONE OPERAND ¬ Not or …

¬ x ¬ Logical expression

Slide 36 — Logical Notations: TWO OPERANDS

• x ∧ y (conjunction with two conjuncts)
• x ∨ y (disjunction, and its operands are called disjuncts.)
• x ⇒ y (implication, the antecedent ⇒ the consequent)
• x ⇐ y pronounced x is implied by y.
the consequent⇐ the antecedent.)

Slide 37 — Logical Notations: TWO OPERANDS

• x ∧ y minimum
• x ∨ y maximum
• x ⇒ y Implication is the ordering operator.
It means:
• "lower than or equal to". OR “x is stronger than or equal to y.”
• x ⇐ y a reverse implication is an ordering operator. It means
“higher than or equal to". OR “x is weaker than or equal to y.”

Slide 38

Logical Notations: TWO OPERANDS CONT.
• x = y (It's an expression whose result is true or false.
• x ≠ y (x is unequal to y.)

Slide 39 — Logical Notations: THREE OPERANDS

• [It's called conditional composition, or if-then-else-fi, and the operands are called the if-part, the then-part, and the else part]
if x then a else b fi
• Aprecedence table for order of evaluation is in the H/O, P. 246.
• Whenever you want some other order of valuation, use parentheses, (round brackets).
• On that same table and in P. 234 it define operators’ associativity

Slide 40 — Characteristics of Logical Operators

(No body text on this slide.)

Slide 41 — Logical Notations ASSOCIATIVITY

• Conjunction and disjunctions are associative, for example:
• Conjuntion:
a∧ b∧ c ≡ a∧ b ∧ c
• Disjunction.
a∨ b∨ c ≡ a∨ b ∨ c

Slide 42 — ≡ (a ⇒ b)

• Implication, equal and unequal connectors are both associative and continuing operators.
• a ⇒ b ⇒ c ≡ a ⇒ (b ⇒ c)
⇒ c ≡ (a ⇒ b)

∧ (b ⇒ c)
≡ (b ⇒ c)
∧ (a ⇒ b)
• a = b= c ≡ b=a ∧ c=b ≡ a=b ∧ b=c;
a ≠ b ≠ c ≡ a ≠ b ∧ b ≠ c
• As you notice, no need of parentheses to write a conjunction of equations, Yet, conjunction of implications needed parentheses.
Why?

Slide 43 — EVALUATION OF BINARY NOTATIONS

TT T⊥ ⊥ T ⊥⊥
∧ T ⊥ ⊥ ⊥
T ⊥
∨ T T T ⊥
¬ ⊥ T
⇒ T ⊥ T T
⇐ T T ⊥ T
= T ⊥ ⊥ T
≠ ⊥ T T ⊥
TTT TT⊥ T⊥ T T⊥⊥ ⊥ TT ⊥ T⊥ ⊥⊥ T ⊥⊥⊥
if x then a else b fi T T ⊥ ⊥ T ⊥ T ⊥
x ⇒ a ∧ ¬ x ⇒ b ≡ x ∧ a ∨ ¬ x ∧ b

Slide 44 — Substitution (Instantiation)

• Given a conjunction x∧ y, Substitution stand for all expressions obtained by replacing the variables x and y with arbitrary binary expressions.
• Example, we might:
• replace x by (⊥⇒ ¬(⊥∨⊤)) and replace y by (⊥∨⊤)
Get the conjunction (⊥⇒ ¬(⊥∨⊤))∧ (⊥∨⊤).
• Replacing a variable with an expression is called substitution or instantiation.

Slide 45 — Applying Binary Rules To An Application:

the grass is green,
Theorem Antitheorem the sky is green, there's life elsewhere in the universe, Not a Theorem nor Antitheorem intelligent messages are coming from space Not a Theorem nor Antitheorem
Theorem Not a Theorem nor Antitheorem In number theory: 1 + 1 = 2 0 /0 = 5
➢ The application, also, must classify (which is theorem, and which is antitheorem) those new expressions.
➢ When classifying binary expressions as theorems and antitheorems:
• Be consistent (don't classify some binary expression as both a theorem and an antitheorem.)
• It is a choice to be complete, but you MUST be consistent.

Slide 46 — Classification of Theorem

• the binary expression 1 + 1 = 2, is classified as a theorem.
• The binary expression 1 + 1 > 2, is classified as antitheorem.
• But it is perfectly legitimate to leave a binary expression unclassified. (yet your application will be incomplete)
• For example, 1 /0 = 5 will be neither a theorem nor an antitheorem.
• An unclassified binary expression may correspond to a statement whose truth or falsity we do not know or do not care about.

Slide 47 — Consistency Theorem

• Atheory is called consistent if NO binary expression (exp) is both a
theorem and an antitheorem.

(exp=T ∧¬ exp= ⊥∨ exp= ⊥∧¬ exp= T)
≡ (exp=T ∧ exp ≠⊥ ∨ exp= ⊥ ∧ exp ≠ T)
Consistency is an ESSENTIAL PROPERTY for any binary expression.
Ex:{ exp ∧ ¬ exp is always antitheorem, while exp ∨ ¬ exp is always a
theorem} Consistent
• Atheory is inconsistent if some binary expression is both a theorem and an antitheorem.

Slide 48 — Example:

• Atheory is INCONSISTENT if some binary expression is both a theorem and an antitheorem.
p = x ⇒ (x∧ ¬ x)
assume x = T (Theorem) then p= T ⇒ (T∧ ¬ T) ≡ ⊥
(Antitheorem)
assume x = ⊥ (Antitheorem) then p= ⊥ ⇒ (⊥∧ ¬ ⊥) ≡ T (Theorem)
THEN the expression p = x ⇒ (x∧ ¬ x) is inconsistent

Slide 49 — EXAMPLE

• Two or more logical expression are logically consistent if it is possible for them all to be true at the same time. If there is no way for them all to be true at once, they are inconsistent. Inconsistent logical expressions are said to contradict one another.
• These three expressions a ⇒ b, a ∨ b, ¬ a are consistence
• While those two (A ∧ B), (¬ A ∨ ¬ B) are inconsistence

Slide 50

A B (A ⇒ B) (A ∨ B) ¬ A 0 0 1 0 1 0 1 1 1 1 Consistence 1 0 0 1 0 1 1 1 1 0 A B (A ∧ B) (¬ A ∨ ¬ B)
0 0 0 1 Inconsistence 0 1 0 1 1 0 0 1 1 1 1 0

Slide 51 — Com plete Theorem

• Atheory is called complete if every fully instantiated binary expression is either a theorem or an antitheorem, (T ∨⊥)
• A + b = c instantiated to 1+ 2 = 3 ≡ T c= 3 instantiated to 1+ 2 = 5 ≡ ⊥ c= 5
• Atheory is incomplete if some fully instantiated binary expression has no classification (underdetermined, unprovable to be either a Theorem or an Antitheorem).
• x= 0/0; 5= 1/0 (both 0/0 and 1/0 are undefined)

Slide 52

Example: Given the four statements.
(i) This statement is true.
(ii) This statement is false.
(iii) This statement is either true or false.
(iv) This statement is neither true nor false.
Which of these statements are (a) true? (b) false? (c) neither true nor false (underdetermined, incomplete)? (d) both true and false (overdetermined, inconsistent)?

Slide 53

(i) This statement is true.
i≡ i=T the sol is a value of i that make it true
ASSUME i To BE Theorem (TRUE) THEN T ≡ T=T ≡ T
ASSUME i To BE ANTITHEREM ⊥ ≡ (⊥ =T) ≡ T
TWO Solution (Incomplete)
----------(ii) This statement is false.
ii≡ ii=⊥ the sol is a value of ii that make it true

ASSUME ii To BE Theorem (TRUE) THEN T ≡ (T= ⊥) ≡ ⊥
ASSUME ii To BE ANTITHEREM ⊥ ≡ ⊥ = ⊥ ≡ ⊥
NO Solution (inconsistent)
UNIVERSITY

Slide 54

(iii) This statement is either true or false.
iii ≡ iii=T ∨ iii=⊥ the sol is a value of iii that make it true
ASSUME iii To BE Theorem (TRUE) THEN T ≡ T=T ∨ T= ⊥ ≡ T
ASSUME iii To BE ANTITHEREM ⊥ ≡ (⊥ =T ∨ ⊥ = ⊥)= T ≡ ⊥
Ture is the value of iii that makes the expression true
----------(iv) This statement is neither true nor false.
iv≡ iv=¬ T ∧ iv=¬⊥ the sol is a value of iv that make it true
ASSUME iv To BE Theorem (TRUE) THEN T ≡ (T=¬ T∧ T= ¬⊥)= ⊥ ≡ ⊥
ASSUME iv To BE ANTITHEREM
⊥ ≡ (⊥ = ¬ T∧⊥ = ¬⊥) = ⊥ ≡ T
False is the value of iv that makes the expression true
UNIVERSITY

Slide 55

Thie following approach may be used when the LHS is not the only variable in the expression Analyze (i) P = This statement is true P ≡ S=T
ASSUME S To BE Theorem (TRUE) THEN P ≡ T=T ≡ T
ASSUME S To BE ANTITHEREM P ≡ ⊥ =T ≡ ⊥
(i) is either true or false underdetermined, incomplete

Slide 56

Analyze (ii) P= This statement is false.
P= S= ⊥
ASSUME S TO BE Theorem (TRUE) THEN P ≡ T= ⊥ ≡ ⊥ neither true
ASSUME S TO BE ANTITHEREM P ≡ ⊥ = ⊥ ≡ T nor false is neither true nor false ≡ Inconsistence

Slide 57

Analyze (iii) P = This statement is either true or false.
P = S=T or S= ⊥
ASSUME S TO BE Theorem (TRUE) THEN P ≡ T= T ∨ T= ⊥ ≡ T TRUE
ASSUME S TO BE ANTITHEREM P ≡ ⊥ = T ∨ ⊥ = ⊥ ≡ T TRUE
TRUE

Slide 58

Analyze (iv) P =This statement is neither true nor false.
P= S= ¬ T and S= ¬⊥
ASSUME S TO BE Theorem (TRUE) THEN P ≡ T= ¬ T ∧ T= ¬⊥ ≡ ⊥ FALSE
ASSUME S TO BE ANTITHEREM P ≡ ⊥ = ¬ T ∧ ⊥ = ¬⊥ ≡ ⊥ FALSE False

Slide 59

• Answer (a) §Statement (iii) is true.
(b) §Statement (iv) is false.
(c) §Statement (i) is either true or false, (underdetermined, incomplete).
(d) §Statement (ii) is neither true nor false (overdetermined, inconsistent).

Slide 60 — S Exp Class

HERE IS
HOW ⊥
⊥ Incomplete E xp = F(S)
T T ⊥
T Inconsistent T ⊥
⊥ ⊥ ⊥
T ⊥ ⊥
T T T
T

Slide 61 — Theorem Proving: Proof Rules

• Finding out if a binary expression is a theorem or antitheorem is called
PROVING.
Five rules for proving 1. Axiom Rule: If a binary expression is an axiom, then it is a
theorem. If a binary expression is an Antiaxiom, then it is an antitheorem.
2. Evaluation Rule: If all the binary subexpressions of a binary expression are classified, then it is classified according to the truth tables.

Slide 62

Theorem Proving: Proof Rules 3. Completion Rule: If a binary expression contains unclassified binary subexpressions, and all ways of classifying them place it in the same class, then it (expression) is in that class.
4. Consistency Rule: If a classified binary expression contains unclassified binary subexpressions, and only one way of classifying them is consistent, then they are classified that way.
5. Instance Rule: If a binary expression is classified, then all its instances have that same classification.

Slide 63 — First Proof Rule: Axiom

• An axiom is a binary expression that is stated to be a theorem.
• An anti-axiom is similarly a binary expression stated to be an antitheorem.
• the grass is green, Axiom
• the sky is green, an Antiaxiom
• there's life elsewhere in the universe, Unclassified
• intelligent messages are coming from space, Unclassified
• intelligent messages are coming from space ⇒ there is life elsewhere Axiom

Slide 64 — First Proof Rule: Axiom

• Axiom: ⊤
Theorem
• Anti-axiom: ⊥
Antitheorem
• Axiom: grass is green
Theorem
• Anti-axiom: sky is green Antitheorem
• Axiom: (intelligent messages are coming from space)
⇒ (there is a life elsewhere in the universe)
Theorem

Slide 65

Second Proof Rule: The Evaluation If all the subexpressions are classified, then use the truth tables.
• If you know what the operands are, then the truth tables tell you what the whole expression is.

Slide 66 — Third Proof Rule: The Com pletion

• If an expression contains unclassified subexpressions, and all ways of classifying them place it (expression) in the same class, then it is in that class.
• If you don't know what the operands are, you might still be able to tell what the whole expression is.
• Example: Theorem: there is life elsewhere in the universe ∨ T.
if there's life elsewhere, then that's T ∨ T ≡ T.
If there isn't life elsewhere, then that's ⊥ ∨ T ≡ T.
It's a theorem by completion rule

Slide 67 — Third Proof Rule: The Completion Examples:

Theorem: (a) ∨ ¬ (a) because T ∨ ⊥ = ⊥ ∨ T = T
• Antitheorem: p = (a) ∧ ¬ (a)
Assume a= T then p = T ∧ ⊥ = ⊥
Assume a= ⊥ then p = ⊥ ∧ T = ⊥

Slide 68 — Fourth Proof Rule: The Consistency

• If a classified binary expression contains binary subexpressions, and only one way of classifying them is consistent, then they are classified that way.
• This time, we know what the whole expression is, and we're wondering
what the parts might be.
Example 1: x, x ⇒ y are theorems, what is y?
Assume y is an antitheorem,
x ⇒ ⊥ ≡ T ⇒ ⊥ ≡
¬ T
∨⊥ ≡ ⊥ ∨ ⊥ ≡ ⊥
i.e., x ⇒ y is antitheorem contradicting the premises therefore, y is not antitheorem, i.e., y is a theorem

Slide 69 — Example 2:

Theorem: ¬ x, What's x?
• Assume x = T i.e., assume x is a theorem
• Apply the evaluation rule, ¬ x = ⊥ is an antitheorem, contradicting that (¬ x) is a theorem.
• So x has to be an antitheorem.
• This example means no need to talk about Antiaxioms or antitheorems.
• If we want to say something is an antitheorem, we just say its negation is a theorem.
• (We also don't need to have both true and false)

Slide 70

Fifth Proof Rule: The Instance If a binary expression is classified as a theorem or an antitheorem, then all its instances have that same classification.
axiom: x = x. Theorem: x = x
Theorem: (T= ⊥∨⊥) ≡ (T= ⊥∨⊥)

Slide 71 — Classification of Logical Systems

• Classical logic: all five rules, (which we are using in this course)
• Constructive logic: don't allow the completion. In a logic course, it's interesting to see what you can do if we DO NOT allow completion rule. (do not try several states, one may only substitute the known values of the subexp., and/or apply consistency rule)
• Evaluation logic: don't allow the consistency rule and the completion rule.

Slide 72 — EXPRESSION and PROOF FORMAT

• x∧ y ∨ z leave more space around operators with less precedence x ∧ y∨ z is misleading, and causes errors.
• When an expression is too long to fit on one line, it should be broken at a sensible place, which usually means at the main connective.
(First part)
∧ (Second part)
• Put left brackets on the left, and right brackets on the right.
• Here's a continuing equation, nicely formatted.
Expression 0 Expression 0= Expression 1 means
= Expression 1 ∧ Expression 1= Expression 2
= Expression 2 ∧ Expression 2= Expression 3
= Expression 3

Slide 73 — Expression and Proof Form at

• Do Do Not
• x∧ y ∨ z x ∧ y∨ z (First (First part)
part) ∧ (Second ∧ (Second part)
part)
Expression 0 Hint 0 (tells why expression 0 equal expression
= Expression 1 Hint1
= Expression 2 Hint2
= Expression 3

Slide 74 — Expression and Proof Form at: Exam ple

• Example: Prove: a∧ b ⇒ c ≡ a ⇒ (b ⇒ c)
§ a ∧ b ⇒ c Material implication ≡
¬ (a∧ b)
∨ c Duality ≡ ¬ a ∨ ¬ b ∨ c Material implication ≡ a ⇒ ¬ b ∨ c Material implication ≡ a ⇒ (b ⇒ c)
Material implication: x ⇒ y =
¬ x ∨ y a ∧ b ⇒ c = ¬ (a ∧ b)
∨ c

Slide 75 — Proof of Expression Format: Example

• Simplify to true: a ∧ b ⇒ c ≡ a ⇒ (b ⇒ c)
§ a ∧ b ⇒ c ≡ a ⇒ (b ⇒ c) Material implication 3 times = (
¬ (a ∧ b)
∨ c ≡ ¬ a ∨ (¬ b ∨ c)) Duality = (
¬ a ∨¬ b ∨ c ≡ ¬ a ∨ ¬ b ∨ c) Reflexivity of ≡
= T

Slide 76

Read The Handout Carefully, To Realize The Precedence and The Rules.
You Should Know The Logic and Numerals Laws.
Memorize As Much, or at Least Know Where To Find Them Solving the Assignments are essential for Training understanding and passing the course